Midterm Problem 1-2

Wen Sheng (ws2840)

2026-10-18

Problem 1

The following code simulates and plots 5000 trajectories of Geometric Brownian motion with starting value $X_0 = 100$, growth rate$\mu = 0.10$, and volatility $\sigma = 0.21$.

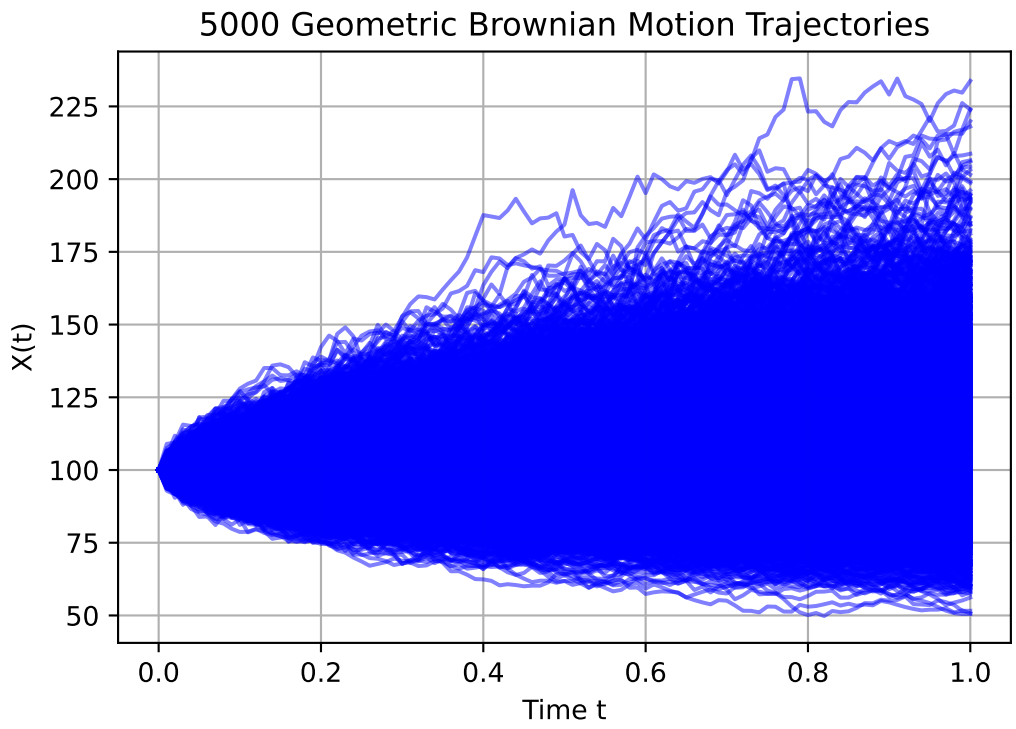
```
import numpy as np
import matplotlib.pyplot as plt

n_paths = 5000
n_steps = 100
T = 1.0
dt = T / n_steps
Sqrtdt=np.sqrt(dt)
rng = np.random.default_rng(17) #initialize rand numb generator
X = np.zeros((n_paths, n_steps+1)) #Allocate a zero array of trajectories

#GBM setup
X0 = 100
mu = 0.10
sigma = 0.21

X[:, 0] = X0 # initial starting point to X0
for i in range(n_paths):
    for j in range(1,n_steps+1):
      z = rng.normal(loc=0, scale=1)
      # GBM: X_(t + dt) = X_t * exp{(mu - 1/2 * sigma^2)dt + sigma * sqrt(dt)z}
      X[i, j] = X[i, j-1] * np.exp((mu - 0.5 * sigma**2) * dt + sigma * Sqrtdt * z)

t = np.linspace(0, T, n_steps + 1)
# Plotting
plt.figure(figsize=(6, 4))
plt.plot(t, X.T, alpha=0.5, color='blue')
plt.title('5000 Geometric Brownian Motion Trajectories')
plt.xlabel('Time t')
plt.ylabel('X(t)')
plt.grid(True)
plt.show()
```



Problem 2

The code plots the Black–Scholes call price with dividend yield $q = 3.2\%$ and strike $K = 11.50$, alongside its expiration payoff.

```
import math
import statistics
import matplotlib.pyplot as plt
import numpy as np

def BlackScholesStocks(callput, S, K, r, sigma, T, q=0.0):
    # d1 = (ln(s/k) + (r-q + 1/2 * sigma^2)T)/(sigma*sqrt(T))
    d1 = (math.log(S / K) + (r - q + 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
    d2 = d1 - sigma * math.sqrt(T)
    if callput == 'c':
        N1 = statistics.NormalDist().cdf(d1)
        N2 = statistics.NormalDist().cdf(d2)
        price = S * math.exp(-q * T) * N1 - K * math.exp(-r * T) * N2
    elif callput == 'p':
        N1 = statistics.NormalDist().cdf(-d1)
        N2 = statistics.NormalDist().cdf(-d2)
        price = K * math.exp(-r * T) * N2 - S * N1 * math.exp(-q * T)
    else:
        raise ValueError("callput argument must be 'c' for call or 'p' for put")
    return price
```

```
dx = 0.1  # step to evaluate option for graphing
maxX = 20.0  # max value of Stock price on graph
X = np.arange(dx, maxX + dx, dx)

Strike = 11.5 #update K=11.5
q = 0.032 # dividend = 3.2%
Rate = 0.01
Time = 1.0
Volatility = 0.3
Y = np.maximum(X - Strike, 0)  # array of Call payoff at expiration
Z = np.zeros_like(X)
for i, x_val in enumerate(X):
    Z[i] = BlackScholesStocks("c", x_val, Strike, Rate, Volatility, Time, q=q)

plt.figure(figsize=(6, 4))
plt.title("Black-Scholes Call Price with Continuous Dividends (K=11.50, q=3.2%)")
plt.plot(X, Z, label="Black-Scholes Call Price")
plt.plot(X, Y, label="Payoff at Expiration", linestyle="--")
plt.xlabel("Stock Price")
plt.ylabel("Price")
plt.axvline(x=Strike, linestyle='-.', label=f'Strike={Strike}', color='red')
plt.legend()
plt.grid(True)
plt.show()
```

